

Session 2: Machine Learning Fundamentals

What is Machine Learning?

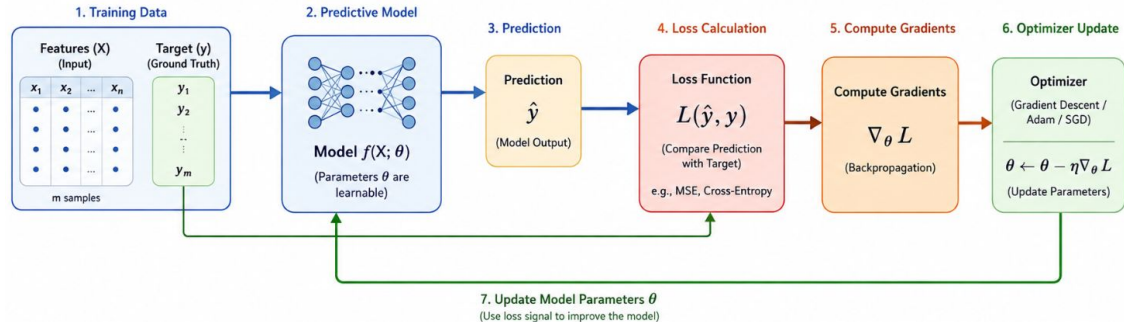
- From rules-based models to Data-Driven models.
- Algorithms learn from data to make predictions (y).
- Model learns a target mapping: $f(x) = y$ ($x \rightarrow$ Input/Features, $y \rightarrow$ Label/Target).
- Difference in prediction (y) and target ($\hat{y}$) is Loss.
- During training, we learn/arrive at values which best mimic our training data.

Simple Linear Regression (one feature like size):

$$y = w_1x_1 + b$$

Multiple Linear Regression (many features like rooms, age, and size):

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$



	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt
0	60	RL	8450	Inside	1Fam	5	2003
1	20	RL	9600	FR2	1Fam	8	1976
2	60	RL	11250	Inside	1Fam	5	2001
3	70	RL	9550	Corner	1Fam	5	1915
4	60	RL	14260	FR2	1Fam	5	2000

X

YearRemodAdd	Exterior1st	BsmtFinSF2	TotalBsmtSF
0	2003	VinylSd	0.0
1	1976	MetalSd	0.0
2	2002	VinylSd	0.0
3	1970	Wd Sdng	0.0
4	2000	VinylSd	0.0

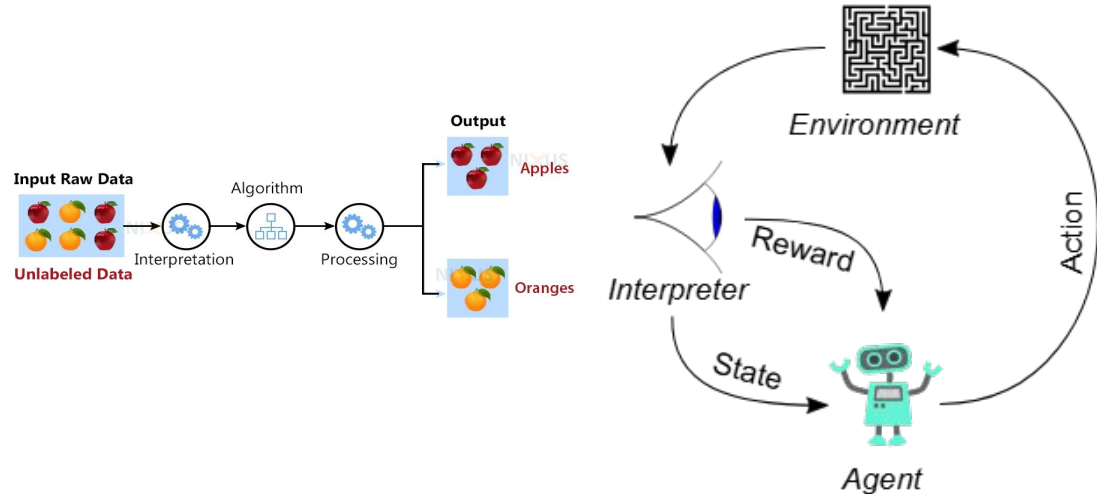
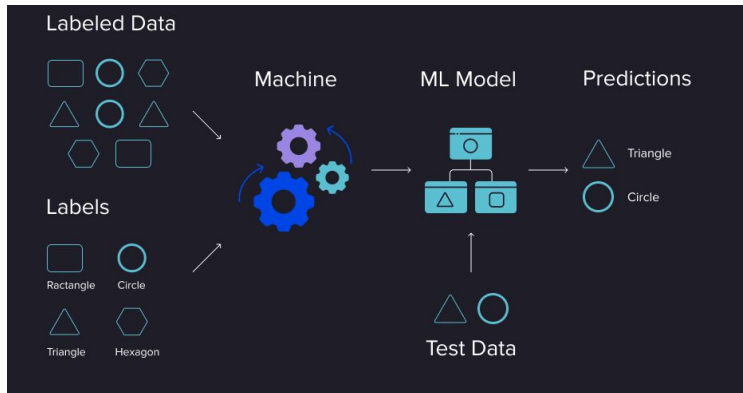
y

208500.0
181500.0
223500.0
140000.0
250000.0

Housing Price Prediction Dataset

Types of ML

- Supervised Learning: We learn the mapping from features (x) $\rightarrow$ Target ($\hat{y}$) from a well-labelled dataset.
- Unsupervised Learning: Learning latent structures from unlabelled data.
- Reinforcement Learning: Model learns a task by interacting with the environment.

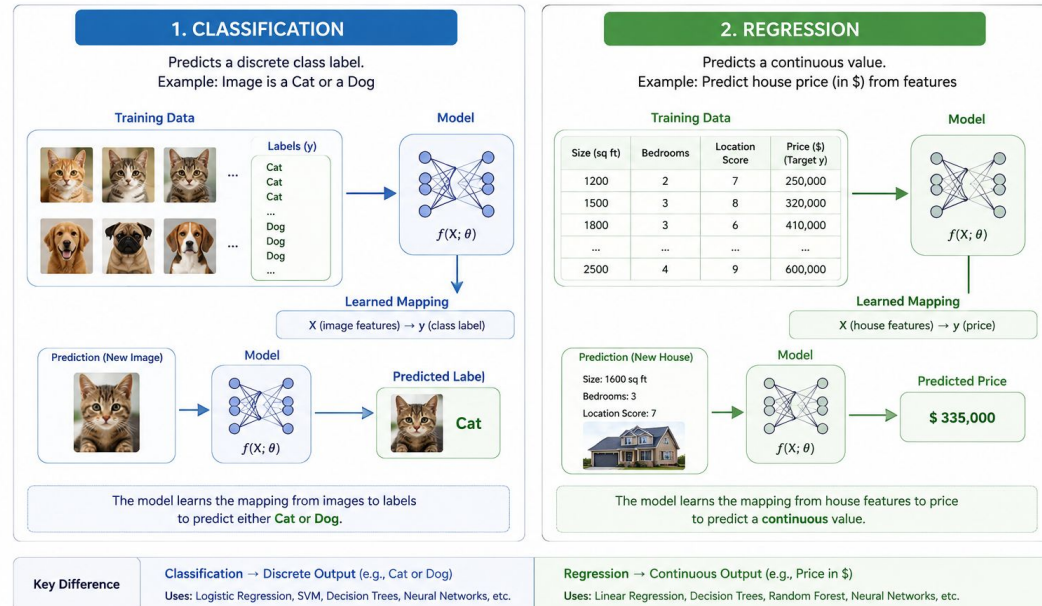
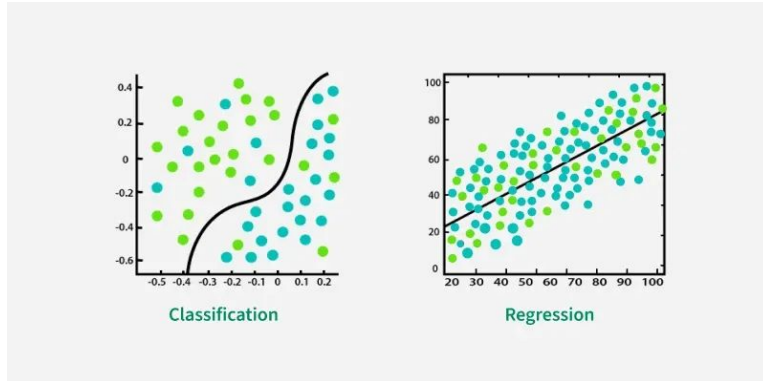


Supervised Learning

- Classification: Predict the class/category of input.
- Regression: Predict a specific value.

Supervised Learning: Two Types

In supervised learning, the model learns a mapping from input features (X) to target (y) using labeled data.

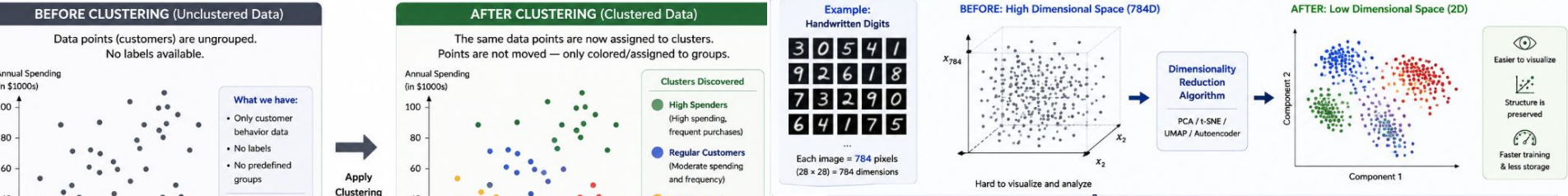


Unsupervised Learning

- Discover hidden patterns, structures or relationship on its own.
- Clustering: Cluster groups data points together based on their similarities.
- Dimensionality Reduction: Learn a mapping to reduce dimension without losing on quality.
- Association Rule Learning: Learn “if-then-else” rules from data.

Unsupervised Learning: Dimensionality Reduction

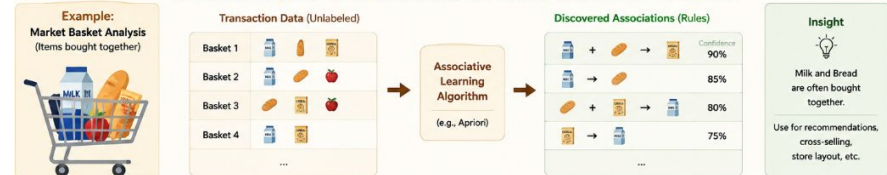
Reduce the number of features while preserving the important information (structure) in the data.



Clustering

Unsupervised Learning: Associative Learning

Discover relationships between items that occur together. (Find associations / co-occurrences in data)

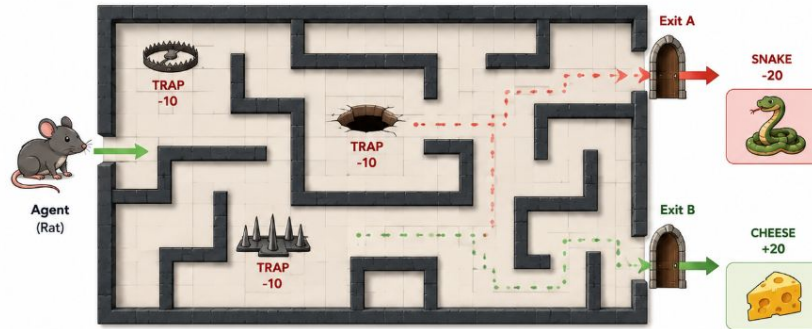


Reinforcement Learning

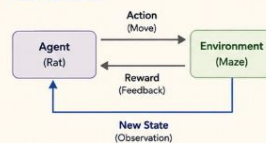
- Give model a task and an environment.
- Model will perform it, evaluate how well it performed -> Give a reward (+1,0,-1).
- Goal of the model is to maximize the cumulative reward, and learn the optimal strategy called policy.

Reinforcement Learning (RL)

The agent learns by interacting with the environment to maximize cumulative reward.



How RL Works



At each step:

1. Agent observes the state (current position)
2. Agent takes an action (up/down/left/right)
3. Environment returns reward (+ / -) and new state
4. Agent learns to choose better actions to get the highest cumulative reward

Legend

- Positive Reward (+)
- Negative Reward (-)
- Agent (Learner)
- Walls / Boundaries
- 🦋 Traps (Negative Reward)

Underfitting vs Overfitting

- We split our data into- train set and test set. Generally 80-20 split.
- Test set reveals true accuracy. Since model has already learnt the train set.
- A model should learn the underlying pattern (representation) of data without memorizing it (otherwise model will fail to generalize).

Underfitting (High Bias)

Model is too simple

Fails to learn the true pattern

High training error

High testing error

Poor performance on both train and test data

Overfitting (High Variance)

Model is too complex

Learns both the pattern and the noise

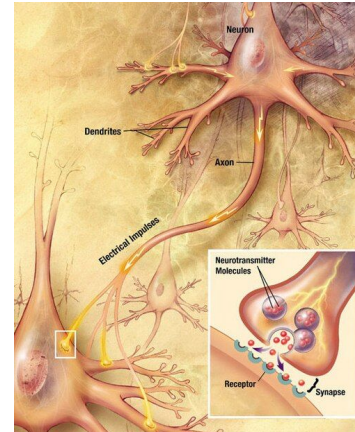
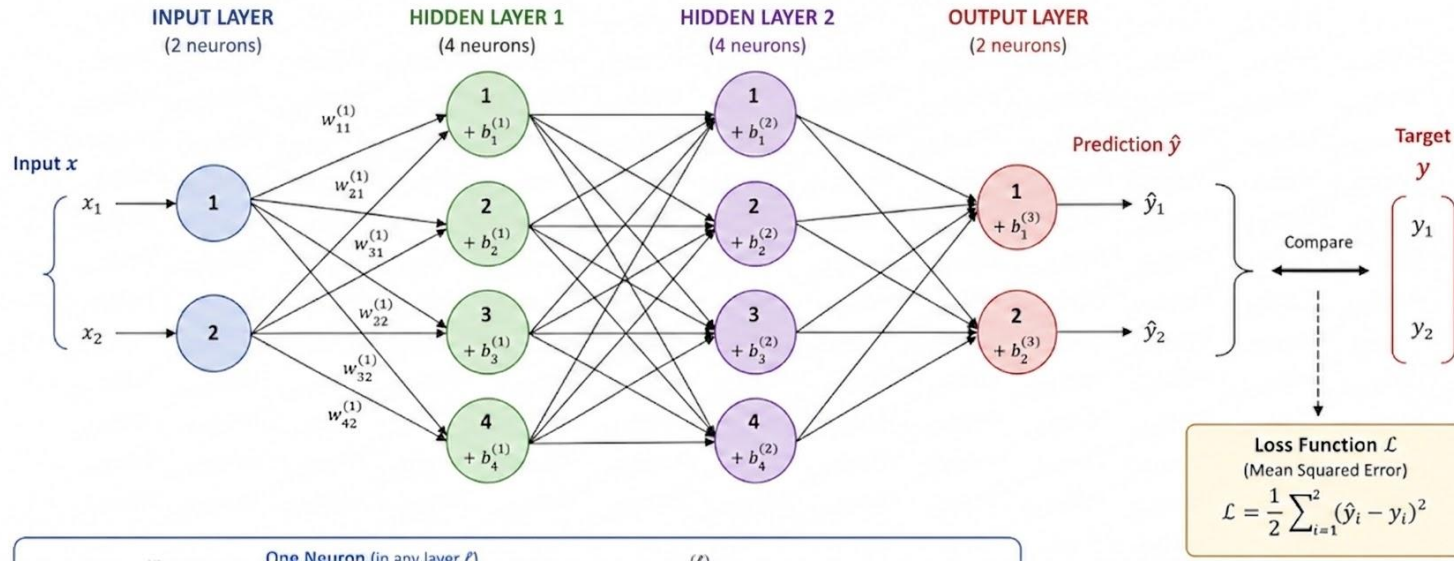
Very low training error

High testing error

Excellent on training data, poor on unseen data

Introduction to Neural Networks

- Inspired from neurons in human body.
- Each perceptron is a linear equation which learns some representation of data.
- All perceptrons are interconnected and pass information in one way.



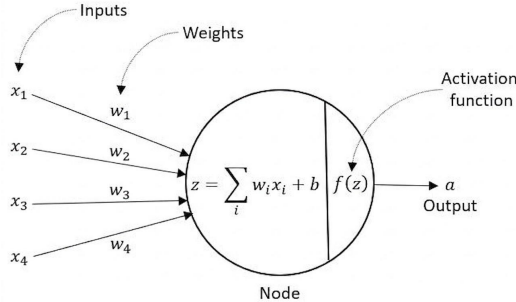
Neural Network Math

A neuron first performs a **linear transformation** of its inputs.

$$z = Wx + b$$

where:

- x = input vector
- W = weight matrix
- b = bias
- z = weighted sum before activation (also called the **pre-activation**)



$$z = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

$$z = \sum_{i=1}^4 w_i x_i + b$$

$$z = \sum_{i=1}^n w_i x_i + b$$

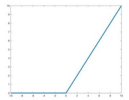
A purely linear network can only learn linear relationships ($y=mx+c$). To capture the complexity of this world, we need to add complexity (non-linear equations). We multiply by a nonlinear function, called an activation function. Its only job is to add non-linearity to the model.

$$a = \phi \left(\sum_{i=1}^n w_i x_i + b \right)$$

Activation function	Equation	Graph
Sigmoid	$S(x) = \frac{1}{1 + e^{-x}}$	
Tanh	$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	

ReLU

$$RELU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$



Leaky ReLU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{otherwise} \end{cases}$$



Demo-

https://youtu.be/3JQ3hYko51Y?si=10_G4PerVaTTHshS&t=47

Backpropagation

- The goal of training is to find those values of weights and bias which gives least loss.
- Gradient descent algorithm is used to update the values of weights and bias.

Hidden Layer 1

For neuron 1,

$$z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$

Activation:

$$a_1^{(1)} = \phi(z_1^{(1)})$$

Hidden Layer 2

Suppose neuron 2 in Layer 2 receives this activation.

$$z_2^{(2)} = w_{21}^{(2)} a_1^{(1)} + w_{22}^{(2)} a_2^{(1)} + \dots + b_2^{(2)}$$

$$a_2^{(2)} = \phi(z_2^{(2)})$$

Output Layer

Suppose output neuron 1 is

$$z_1^{(3)} = w_{11}^{(3)} a_1^{(2)} + w_{12}^{(3)} a_2^{(2)} + \dots + b_1^{(3)}$$

$$\hat{y}_1 = a_1^{(3)} = \phi(z_1^{(3)})$$

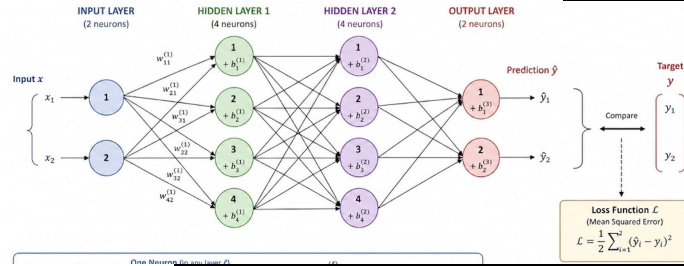
Loss

$$\mathcal{L} = \frac{1}{2} (\hat{y}_1 - y_1)^2$$

Let's say want to update $w_{11}^{(1)}$

$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}}$$

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$



$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}_1} \cdot \frac{\partial \hat{y}_1}{\partial z_1^{(3)}} \cdot \frac{\partial z_1^{(3)}}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \cdot \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \cdot \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}}$$

$$w_{11}^{(1)} \rightarrow z_1^{(1)} \rightarrow a_1^{(1)} \rightarrow z_2^{(2)} \rightarrow a_2^{(2)} \rightarrow z_1^{(3)} \rightarrow \hat{y}_1 \rightarrow \mathcal{L}$$

$$\frac{\partial \mathcal{L}}{\partial w_{11}^{(1)}} = (\hat{y}_1 - y_1) \phi'(z_1^{(3)}) w_{12}^{(3)} \phi'(z_2^{(2)}) w_{21}^{(2)} \phi'(z_1^{(1)}) x_1$$

Gradient Descent

- Using that gradient we update the value of weight.
- Appropriate learning rate (lr) to reach global minima.
- Start with higher lr (converges faster), then reduce lr.
- Backpropagation efficiently computes gradients for every parameter using the chain rule.

The update rule is

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

where:

- θ = any trainable parameter (weight or bias)
- η = learning rate (step size)
- $\mathcal{L}$ = loss function
- $\frac{\partial \mathcal{L}}{\partial \theta}$ = gradient of the loss with respect to the parameter

